

RAI-818: INTELLIGENT **MOBILE ROBOTICS**

Sampling Based Methods

- **Rapidly-Exploring Random Trees (RRTs)**

Text : Principles of Robot Motion – Theory, Algorithms and Implementations
H. Choset, K.M. Lynch, S. Hutchinson, G. Kantor, W. Burgard, et al.

INSTRUCTOR:

DR. KUNWAR FARAZ AHMED KHAN

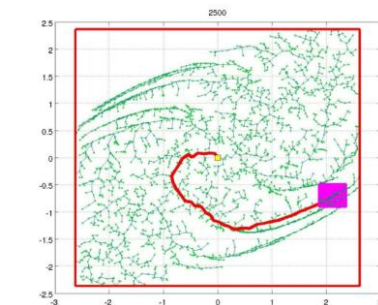
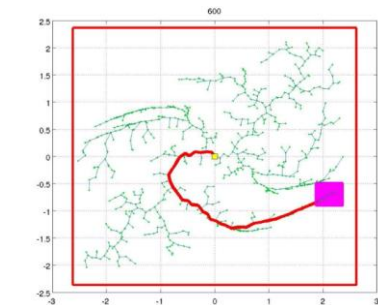
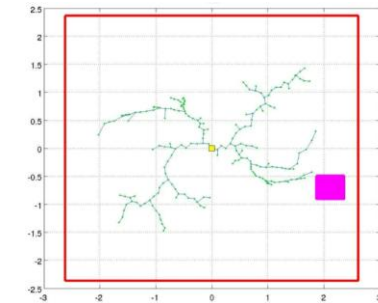
Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation

Tree-Based Motion Planning - Motivation

- Sometimes building a roadmap *a priori* might be inefficient (or even impractical)
 - Assumes that all regions of c-space will be utilized during actual motions
- Building a roadmap requires global knowledge
 - But in real settings, obstacles are not known *a priori*; rather, they are discovered *online*
- We desire an *incremental* method:
 - Generate motion plans for a single start, goal pose
 - Expending more CPU yields better motion plans



Defining futures

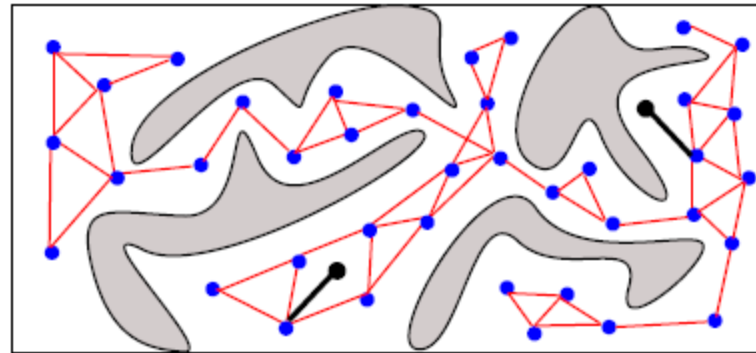
Sampling Based Methods



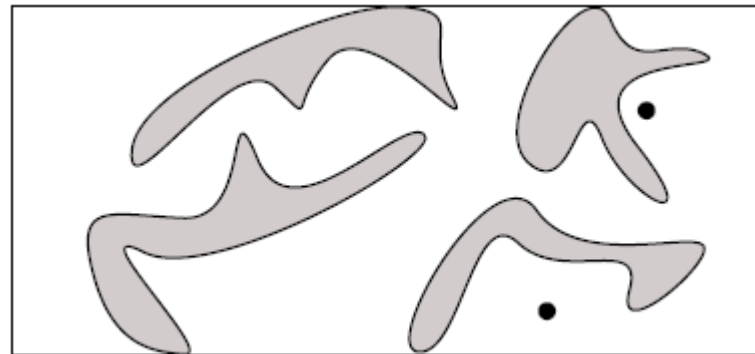
- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation

Tree-Based Motion Planning - Motivation

- PRM-based planners aim to construct a roadmap that captures the whole connectivity of the configuration space



- Good when the objective is to solve *multiple* queries
- May be a bit too much if the objective is to solve a *single* query



Sampling Based Methods



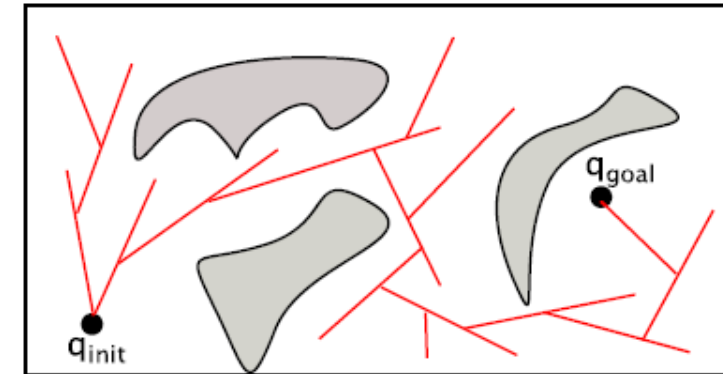
- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - General Idea

General Idea

Grow a tree in free configuration space from q_{init} toward q_{goal}

TREESearchFramework (q_{init}, q_{goal})

- 1: $T \leftarrow \text{ROOTTREE}(q_{init})$
- 2: **while** q_{goal} has not been reached **do**
- 3: $q \leftarrow \text{SELECTCONFIGFROMTREE}(T)$
- 4: $\text{ADDTREEBRANCHFROMCONFIG}(T, q)$



- *Critical Issues*

- How should a configuration be selected from the tree?
- How should a new branch be added to the tree from the selected configuration?



Defining futures

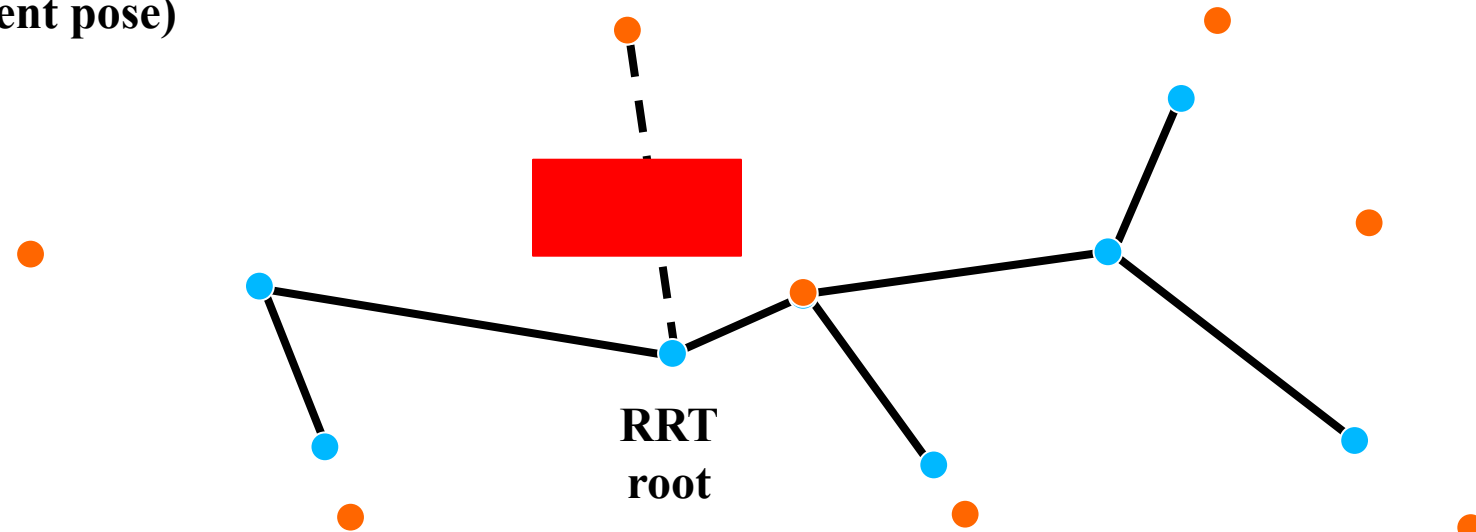
Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - General Idea
 - Algorithm

General Idea - Algorithm

$T = (\text{nodes } V, \text{edges } E)$: tree structure Initialized as single root vertex (i.e., the robot's current pose)



1. $x \leftarrow \text{Sample}()$ // Sample a node x from c-space
2. $v \leftarrow \text{Nearest}(T, x)$ // Find nearest node v in tree
3. $v' \leftarrow \text{Extend}(v, x)$ // Extend nearest node toward sample
4. If $(\text{ObstacleFree}(v, v'))$ then // If extension is collision-free
5. $V \leftarrow V \cup \{v'\}; E \leftarrow E \cup \{(v, v')\}$ // Add new node and edge to tree



Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - General Idea
 - Algorithm

Things to Pay Attention To

```
1  $V \leftarrow \{x_{\text{init}}\}; E \leftarrow \emptyset;$ 
2 for  $i = 1, \dots, n$  do
3    $x_{\text{rand}} \leftarrow \text{SampleFree}_i;$ 
4    $x_{\text{nearest}} \leftarrow \text{Nearest}(G = (V, E), x_{\text{rand}});$ 
5    $x_{\text{new}} \leftarrow \text{Steer}(x_{\text{nearest}}, x_{\text{rand}});$ 
6   if  $\text{ObstacleFree}(x_{\text{nearest}}, x_{\text{new}})$  then
7      $V \leftarrow V \cup \{x_{\text{new}}\}; E \leftarrow E \cup \{(x_{\text{nearest}}, x_{\text{new}})\};$ 
8 return  $G = (V, E);$ 
```

SampleFree() needs to sample a random state from the uniform distribution.

Nearest() searches for nearest neighbor of a given vector. Brute force search examines $|V|$ nodes (increasing)

ObstacleFree() checks the path from the nearest state to the new state for collisions.

Steer() finds the controls that take the nearest state to the new state. Easy for omnidirectional robots. What about non-holonomic systems?



Defining futures

Sampling Based Methods

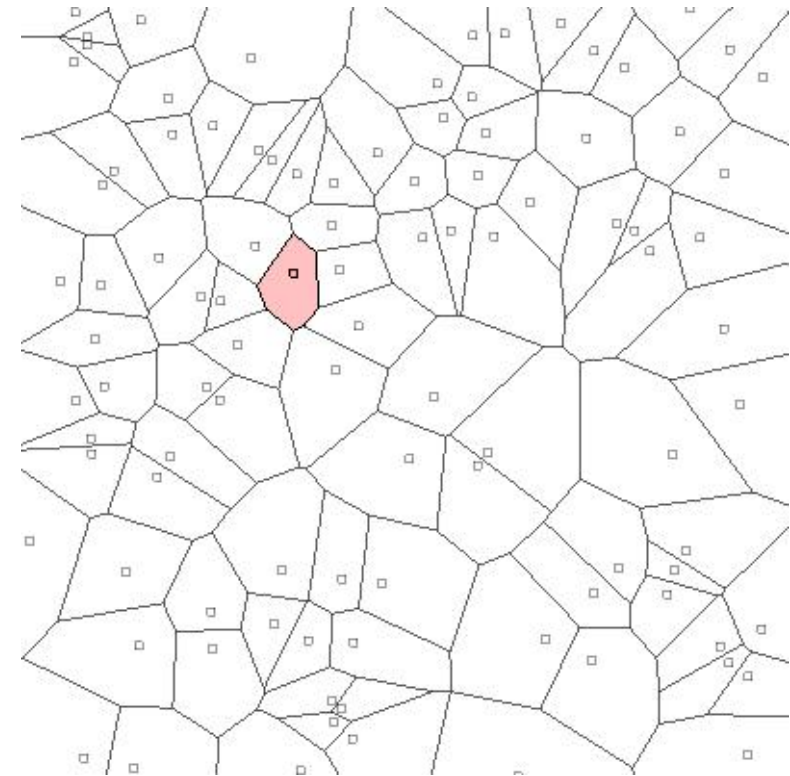


- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - General Idea
 - Algorithm
 - Rapidly Exploring?

Why Rapidly Exploring?

Digression: Voronoi Diagrams

Given n sites in d dimensions, the *Voronoi diagram* of the sites is a *partition* of R^d into regions, one region per site, such that all points in the interior of each region lie closer to that region's site than to any other site



Defining futures

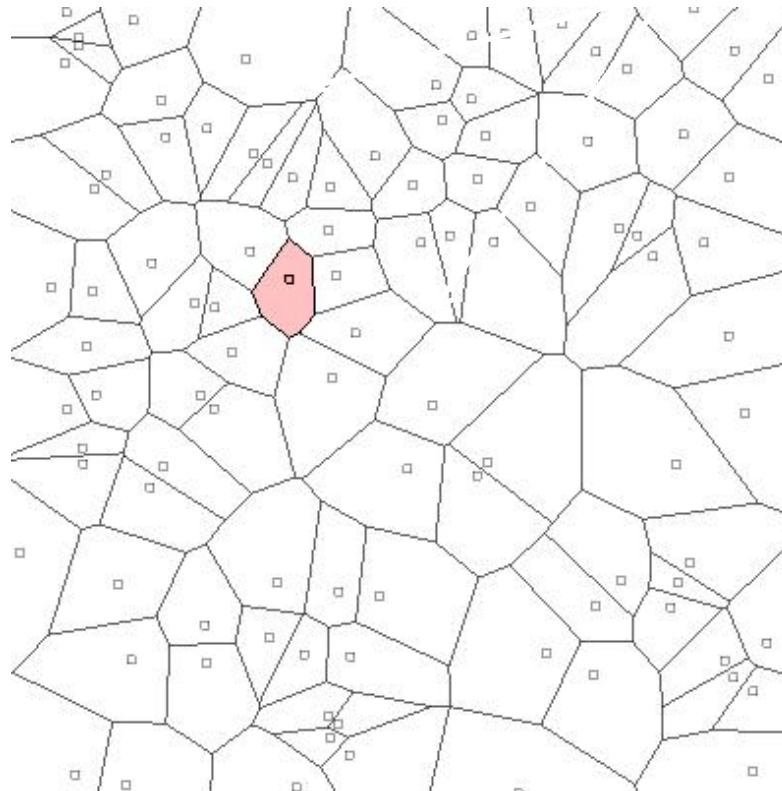
Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - General Idea
 - Algorithm
 - Rapidly Exploring?

Why Rapidly Exploring?

- RRTs tend to grow toward *unexplored* portions of the state-space
 - Unexplored regions are (in some sense) more likely to be sampled
 - This is called a *Voronoi bias*



For an RRT at a given iteration, some nodes are associated with large Voronoi regions of c-space, some with smaller Voronoi regions

The unexplored areas of c-space tend to coincide with the larger Voronoi regions (Uniform) samples will tend to fall into relatively larger Voronoi regions

Thus, unexplored regions will tend to shrink!

Main advantage of RRT: Samples *grow* tree *toward unexplored regions* of C



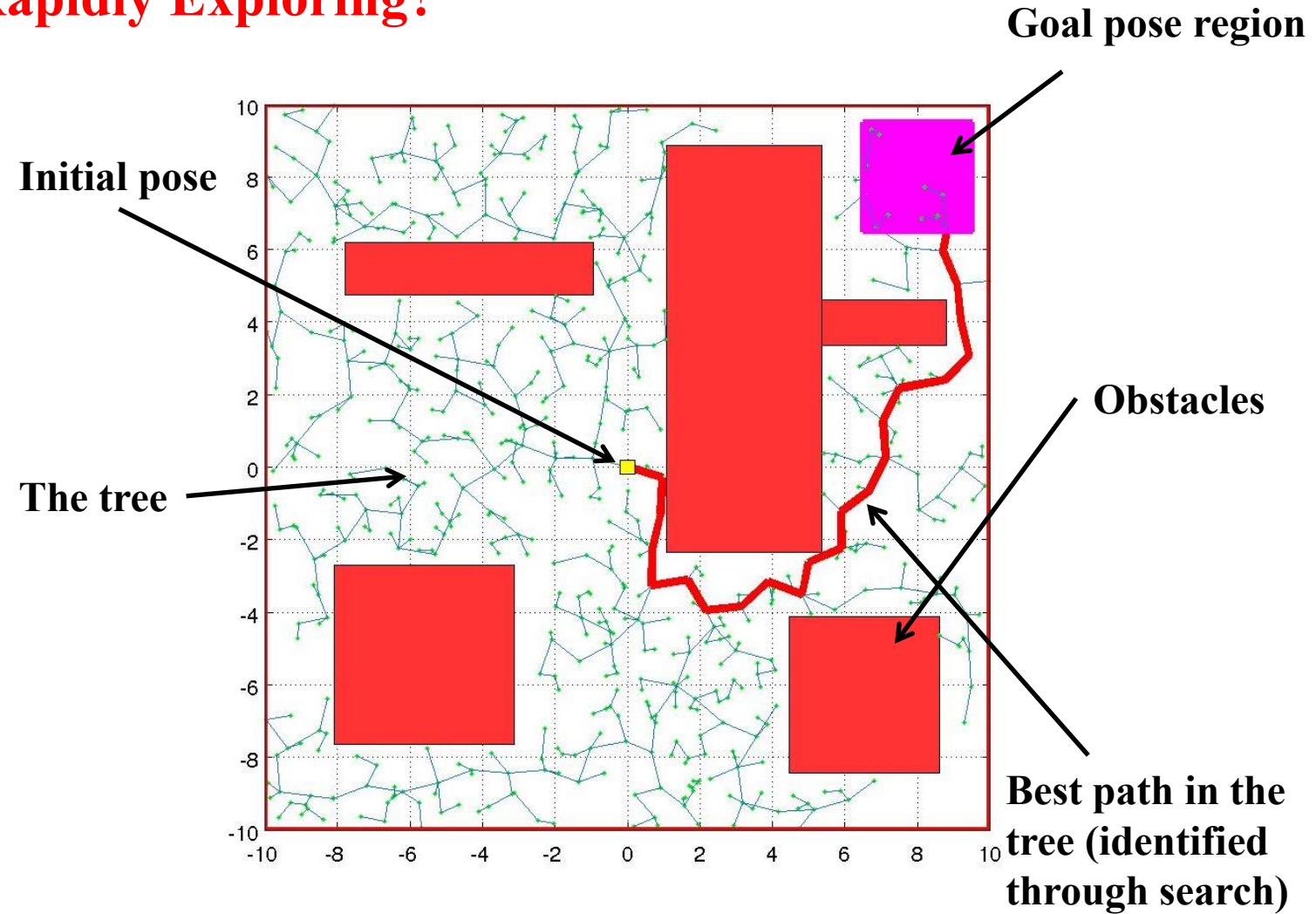
Defining futures

Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - General Idea
 - Algorithm
 - Rapidly Exploring?

Why Rapidly Exploring?



Defining futures

Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Pseudo Code

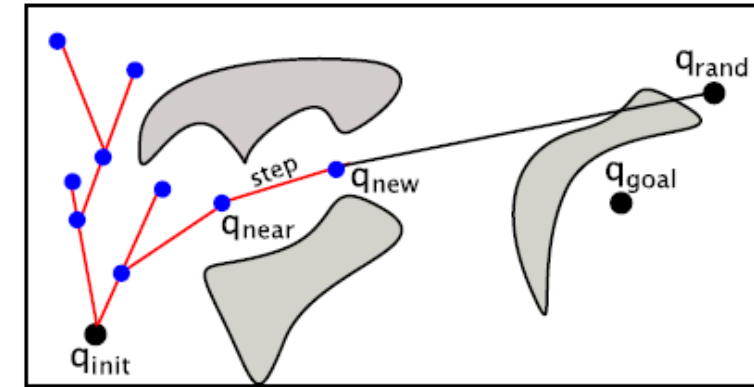
Rapidly-Exploring Random Trees - Code

Pull the tree toward random samples in the configuration space

- **RRT** relies on nearest neighbours and distance metric $\rho: Q \times Q \leftarrow R^{\geq 0}$
- **RRT** adds Voronoi bias to tree growth

RRT (q_{init}, q_{goal})

- **Initialize Tree**
 - 1: $T \leftarrow$ create tree rooted at q_{init}
 - 2: **while** solution not found **do**
- **Select Conguration from Tree**
 - 3: $q_{rand} \leftarrow$ generate a random sample
 - 4: $q_{near} \leftarrow$ nearest configuration in T to q_{rand} according to distance ρ
- **Add New Branch to Tree from Selected Configuration**
 - 5: $path \leftarrow$ generate path (not necessarily collision free) from q_{near} to q_{rand}
 - 6: **if** $ISUBPATHCOLLISIONFREE(path, 0, step)$ **then**
 - 7: $q_{new} \leftarrow path(step)$
 - 8: add config q_{new} and $edge(q_{near}, q_{new})$ to T
- **Check if a Solution is Found**
 - 9: **if** $\rho(q_{new}, q_{goal}) \approx 0$ **then**
 - 10: return solution path from root to q_{new}



Defining futures

Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Pseudo Code
 - GoalBias RRT

Rapidly-Exploring Random Trees

- *Aspects for Improvement*
 - *BasicRRT* does not take advantage of q_{goal}
 - *Tree* is pulled towards random directions based on the uniform sampling of Q
 - In particular, *Tree* growth is not directed towards q_{goal}
- *Suggested Improvements in the Literature*
 - Introduce goal-bias to *Tree* growth (known as *GOALBIASRRT*)
 - q_{rand} is selected as q_{goal} with probability p
 - q_{rand} is selected based on uniform sampling of Q with probability $1 - p$
 - Probability p is commonly set to ≈ 0.05



Defining futures

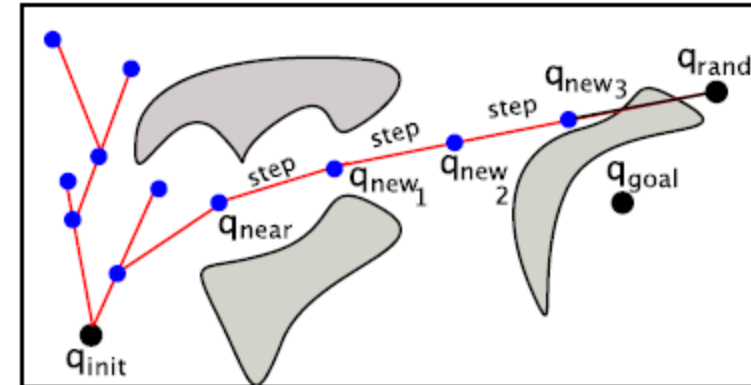
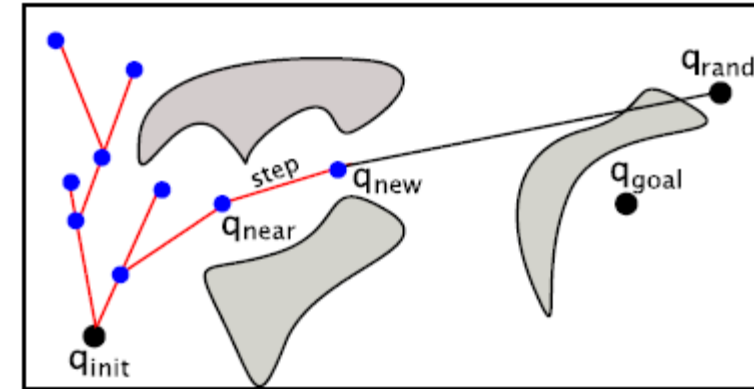
Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Pseudo Code
 - GoalBias RRT
 - Connect RRT

Rapidly-Exploring Random Trees

- *Aspects for Improvement*
 - *BasicRRT* takes only one small step when adding a new Tree branch
 - This slows down *Tree* growth
- Suggested Improvements in the Literature
 - Take several steps until q_{rand} is reached or a collision is found (*CONNECTRRT*)
 - Add all the intermediate nodes to the *Tree*



Defining futures



Sampling Based Methods

- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Pseudo Code
 - GoalBias RRT
 - Connect RRT
 - Expansive Space RRT

Expansive -Space Tree (EST)

Push the tree frontier in the free configuration space

- *EST* relies on a probability distribution to guide *Tree* growth
- *EST* associates a weight $w(q)$ with each *Tree* congfiuration q
- $w(q)$ is a *running estimate* on importance of selecting q as the *Tree* configuration from which to add a new *tree* branch
 - $w(q) = 1 / 1 + \text{deg}(q)$
 - $w(q) = 1 / (1 + \text{number of neighbours near } q)$
 - combination of different strategies
- *SELECTCONFIGFROMTREE*
 - select q in T with probability
$$\frac{w(q)}{\sum_{q' \in V} w(q')}$$
- *ADDTREEBRANCHFROMCONFIG* (T, q)
 - $q_{\text{near}} \leftarrow$ sample a collision-free configuration near q
 - $\text{path} \leftarrow$ generate path from q to q_{near}
 - if path is collision-free, then add q_{near} and (q, q_{near}) to T



Defining futures

Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Pseudo Code
 - GoalBias RRT
 - Connect RRT
 - Expansive Space RRT
 - Higher Dimensions

Observations in High Dimensional Space

- *Tree* generally grows rapidly for the first few thousand iterations
- *Tree* growth afterwards slows down quite significantly
- Large number of configurations increases computational cost
- It becomes increasingly difficult to guide the *Tree* towards previously unexplored parts of the free configuration space

Possible improvements?



Defining futures

Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Pseudo Code
 - GoalBias RRT
 - Connect RRT
 - Expansive Space RRT
 - Higher Dimensions
 - Bi-Direction Trees

Bi-Directional Trees

Grow two trees, rooted at q_{init} and q_{goal} towards each other

- *Bi-Directional Trees* improve computational efficiency over a single **Tree**
- Growth slows down significantly later than when using a single **Tree**
- Fewer configurations in each **Tree** → less of a computational burden
- Each **Tree** explores a different part of the configuration space

BITREE(q_{init}, q_{goal})

- 1: $T_{init} \leftarrow$ create **Tree** rooted at q_{init}
 - 2: $T_{goal} \leftarrow$ create **Tree** rooted at q_{goal}
 - 3: **while** solution not found **do**
 - 4: add new branch to T_{init}
 - 5: add new branch to T_{goal}
 - 6: attempt to connect neighbouring configurations from the two **trees**
 - 7: **if** successful, return **path** from q_{init} to q_{goal}
- Different **Tree** planners can be used to grow each of the **Trees**, e.g., **RRT** can be used for one **Tree** and **EST** can be used for the other



Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Combining RRT & PRM

Combining Roadmap and Tree Based Approaches

- *High-dimensional Motion Planning*
 - **PRM** provides global sampling of the configuration space
 - If sampling is sparse, roadmap is disconnected
 - Dense sampling is impractical in high-dimensions
 - **Trees** provide fast local exploration of area around root
 - **Trees** growth slows down in high-dimensions
 - Bi-Directional trees offer a little improvement
- *Desired Properties for a Motion Planner*
 - Guides exploration towards goal
 - Strikes right balance between breadth and depth of search
- *Sampling-based Roadmap of Trees (SRT)*
 - Hierarchical planner
 - Top level performs global sampling (**PRM-Based**)
 - Bottom level performs local sampling (Tree-Based, e.g., **RRT, EST**)
 - Combines advantages of global and local sampling



Defining futures

Sampling Based Methods



- Sampling Methods

- Probabilistic Tree Approaches

- Motivation
- Implementing RRTs
- Combining RRT & PRM

Sampling Based Roadmap - Trees

CREATETREESINROADMAP

- 1: $V \leftarrow 0; E \leftarrow 0;$
- 2: **while** $|V| < n_{trees}$ **do**
- 3: $T \leftarrow$ create *Tree* rooted at a collision-free config
- 4: use *Tree* planner to grow T for some time
- 5: add T to roadmap vertices V

SELECTWHICHTREESTOCONNECT

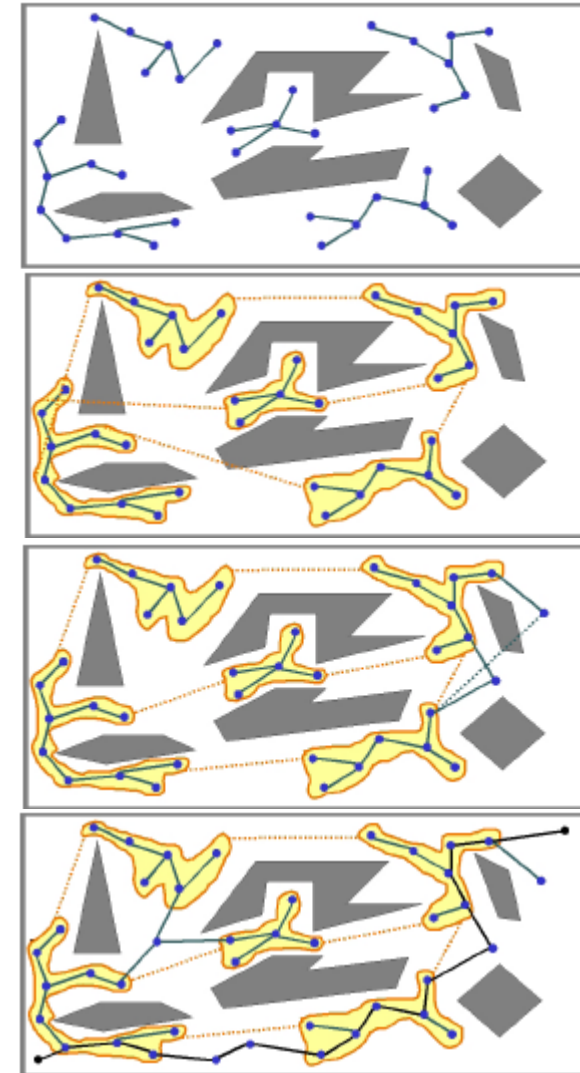
- 1: $E_{pairs} \leftarrow 0;$
- 2: **for each** $T \in V$ **do**
- 3: $S_{neighs} \leftarrow k$ nearest *Trees* in V to T
- 4: $S_{rand} \leftarrow r$ random *Trees* in V
- 5: $E_{pairs} \leftarrow E_{pairs} \cup \{(T, T') : T' \in S_{neighs} \cup S_{rand}\}$

CONNECTTREESINROADMAP

- 1: **for each** $(T_1, T_2) \in E_{pairs}$ **do**
- 2: **if** $ARETREESCONNECTED(T_1, T_2) = false$ **then**
- 3: run *Bi-Directional Tree* planner to connect T_1 to T_2
- 4: **if** connection successful **then**
- 5: add edge (T_1, T_2) to roadmap

SOLVEQUERY(QINIT; QGOAL)

- 1: $T_{init} \leftarrow$ create tree rooted at q_{init}
- 2: $T_{goal} \leftarrow$ create tree rooted at q_{goal}
- 3: connect T_{init} and T_{goal} to roadmap
- 4: search roadmap graph for solution



Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Combining RRT & PRM

Sampling Based Roadmap - Trees

CREATETREESINROADMAP

- 1: $V \leftarrow 0; E \leftarrow 0;$
- 2: **while** $|V| < n_{trees}$ **do**
- 3: $T \leftarrow$ create *Tree* rooted at a collision-free config
- 4: use *Tree* planner to grow T for some time
- 5: add T to roadmap vertices V

SELECTWHICHTREESTOCONNECT

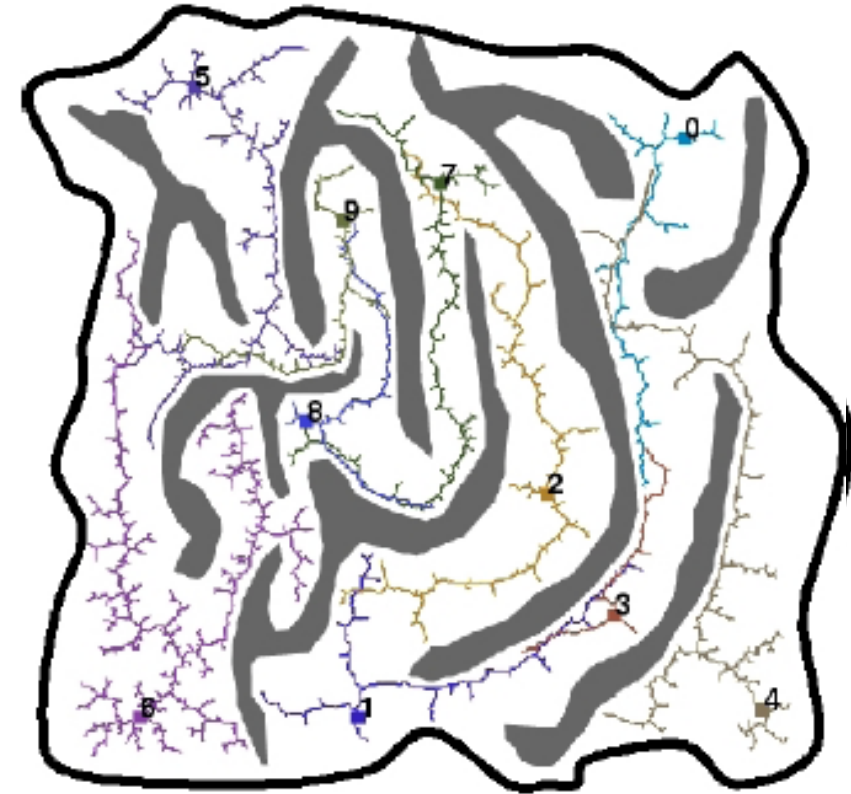
- 1: $E_{pairs} \leftarrow 0;$
- 2: **for each** $T \in V$ **do**
- 3: $S_{neighs} \leftarrow k$ nearest *Trees* in V to T
- 4: $S_{rand} \leftarrow r$ random *Trees* in V
- 5: $E_{pairs} \leftarrow E_{pairs} \cup \{(T, T') : T' \in S_{neighs} \cup S_{rand}\}$

CONNECTTREESINROADMAP

- 1: **for each** $(T_1, T_2) \in E_{pairs}$ **do**
- 2: **if** *ARETREESCONNECTED* $(T_1, T_2) = false$ **then**
- 3: run *Bi-Directional Tree* planner to connect T_1 to T_2
- 4: **if** connection successful **then**
- 5: add edge (T_1, T_2) to roadmap

SOLVEQUERY(QINIT; QGOAL)

- 1: $T_{init} \leftarrow$ create tree rooted at q_{init}
- 2: $T_{goal} \leftarrow$ create tree rooted at q_{goal}
- 3: connect T_{init} and T_{goal} to roadmap
- 4: search roadmap graph for solution



Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Combining RRT & PRM
 - Advantages/Disadvantages

Sampling Based Motion Planning - Summary

- *Advantages*
 - Explores small subset of possibilities by sampling
 - Computationally efficient
 - Solves high-dimensional problems (with 100s of DOFs)
 - Easy to implement
 - Applications in many different areas
- *Disadvantages*
 - Does not guarantee completeness (a complete planner always finds a solution if there exists one, or reports that no solution exists)
- *It offers probabilistic completeness*
 - When a solution exists, it finds a solution with probability as time goes to infinity.
 - When a solution does not exist, it may not be able to determine that a solution does not exist.



Defining futures

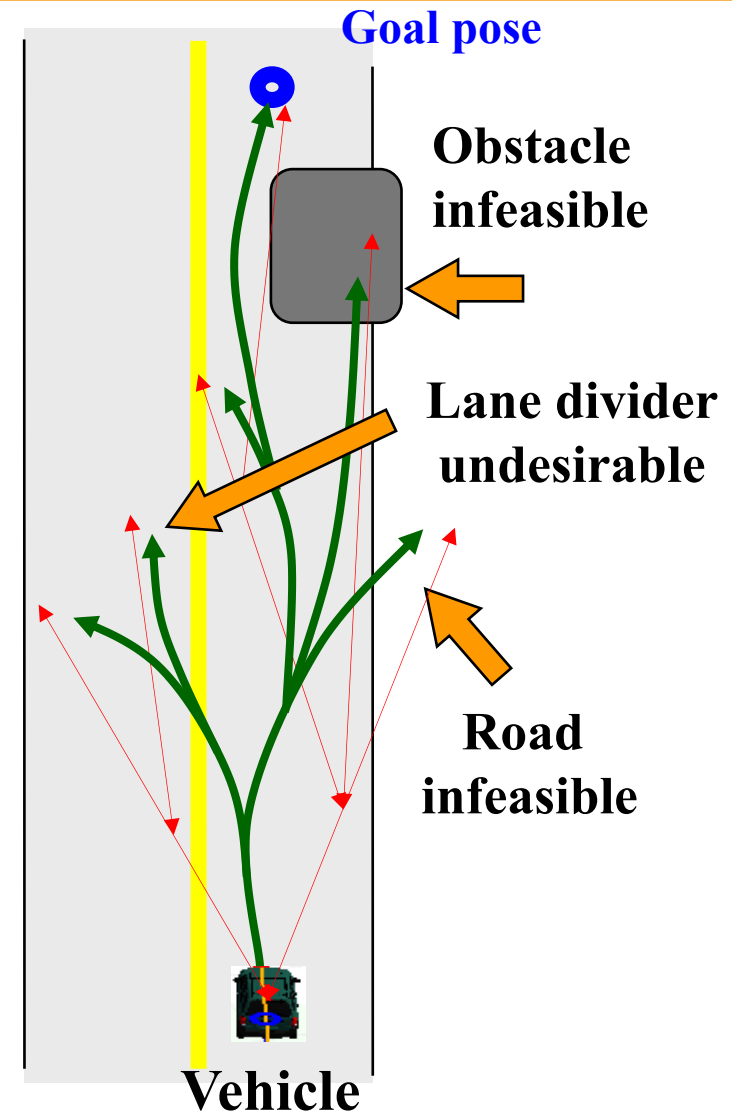
Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Combining RRT & PRM
 - Advantages/Disadvantages
 - Example

Example – Unmanned Driving

- Tree of trajectories is grown by sampling configurations randomly
- Rapidly explores several configurations that the robot can reach.
 - Many test trajectories generated (tens of thousands per second)
- Safety of any trajectory is “guaranteed”...
 - ... as of instantaneous world state at the time of trajectory generation
- Choose best one that reaches the goal, e.g.,
 - Maximizes minimum distance to obstacles
 - Minimizes total path length
- Supports dynamic re-planning; if current trajectory becomes infeasible:
 - Choose another one that is feasible
 - If none remain, then E-stop



Sampling Based Methods



Example – Moving a Piano



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Combining RRT & PRM
 - Advantages/Disadvantages
 - Example



Defining futures

Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Combining RRT & PRM
 - Advantages/Disadvantages
 - Example

Example – Moving a Piano – Connect + Bidirectional



Defining futures

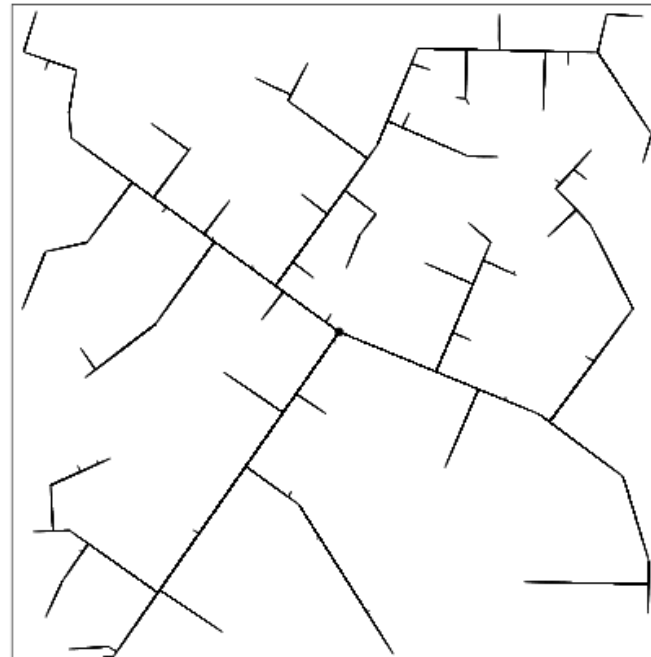
Sampling Based Methods



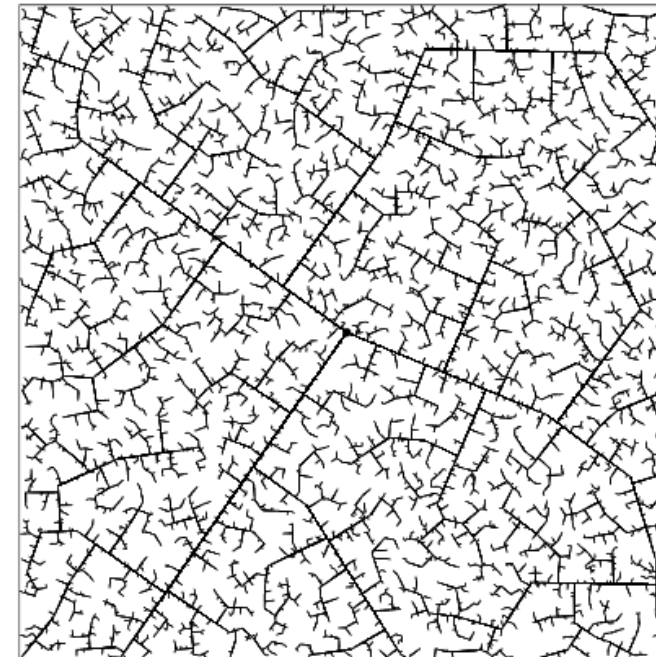
- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Combining RRT & PRM
 - Advantages/Disadvantages
 - Example
 - Summary

Summary – Properties of RRT Algorithm

#1: The RRT will eventually cover the space, i.e. it is a space-filling tree



45 iterations



2345 iterations



Defining futures

Sampling Based Methods

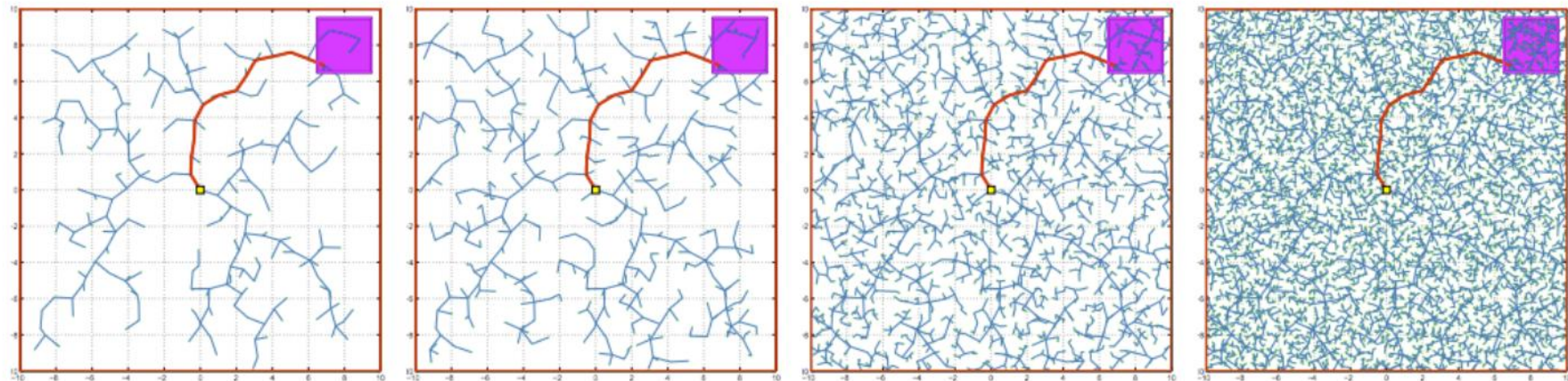


- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Combining RRT & PRM
 - Advantages/Disadvantages
 - Example
 - Summary

Summary – Properties of RRT Algorithm

#1: The RRT will eventually cover the space, i.e. it is a space-filling tree

#2: The RRT will NOT compute the optimal path asymptotically



This problem has been addressed in recent years by RRT*, BIT*, Fast-Marching Trees



Defining futures

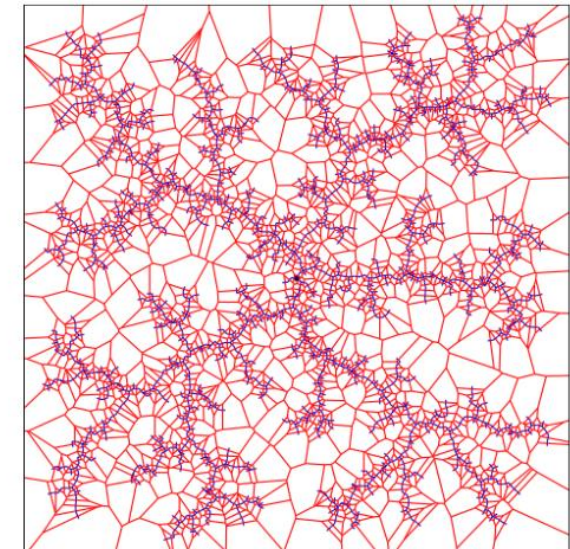
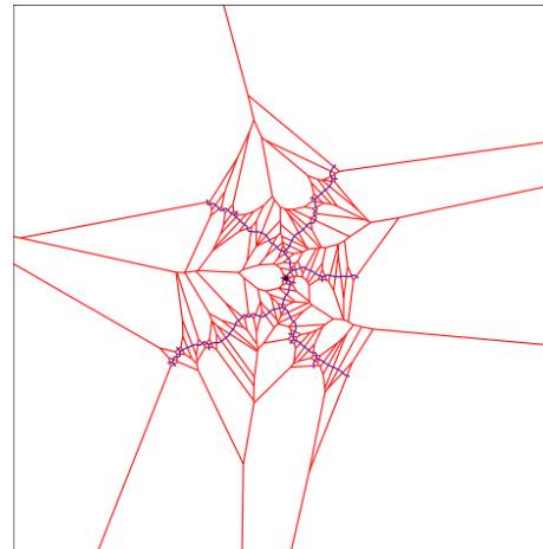
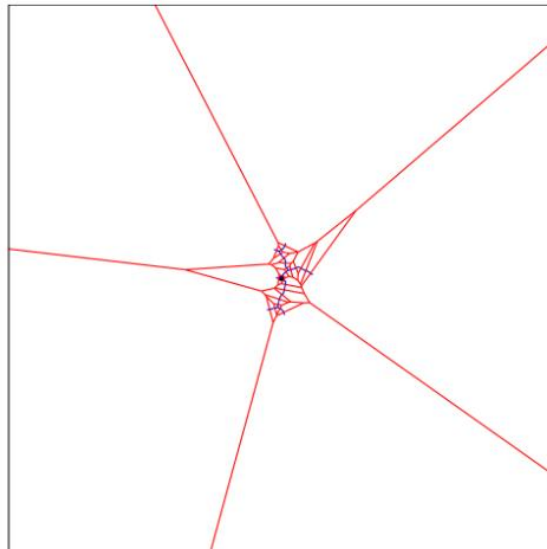
Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Combining RRT & PRM
 - Advantages/Disadvantages
 - Example
 - Summary

Summary – Properties of RRT Algorithm

- #1: The RRT will eventually cover the space, i.e. it is a space-filling tree
- #2: The RRT will NOT compute the optimal path asymptotically
- #3: The RRT will exhibit “Voronoi bias,” i.e. new nodes will fall in free regions of Voronoi diagram (cells consist of points that are closest to a node)



Defining futures

Sampling Based Methods



- Sampling Methods
 - Probabilistic Tree Approaches
 - Motivation
 - Implementing RRTs
 - Combining RRT & PRM
 - Advantages/Disadvantages
 - Example
 - Summary

Summary – Properties of RRT Algorithm

- #1: The RRT will eventually cover the space, i.e. it is a space-filling tree
- #2: The RRT will NOT compute the optimal path asymptotically
- #3: The RRT will exhibit “Voronoi bias,” i.e. new nodes will fall in free regions of Voronoi diagram (cells consist of points that are closest to a node)
- #4: The probability of RRT finding a path increases exponentially in the number of iterations
- #5: The distribution of RRT’s nodes is the same as the distribution used in SampleFree()



Defining futures